

第2章 线性表

目录

2.1线性表及其基本操作

2.1.1 线性表的存储和实现

2.1.2 抽象数据类型描述

2.1.3 线性表顺序存储

2.2 线性表的线性表的基本概念

2.1.2 抽象数据类型描述

2.1.3 线性表顺序存储

2.3 线性表的链式存储和实现

2.1.1 线性表的存储和实现

2.1.2 抽象数据类型描述

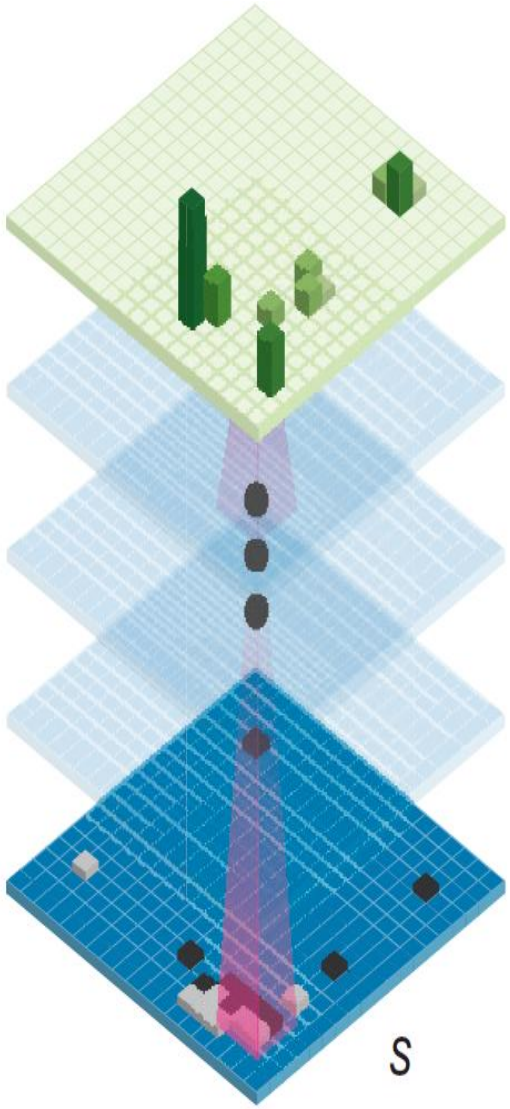
2.1.3 线性表顺序存储

2.4 顺序表与链表的比较

2.1 线性表及其基本操作

- 2.1.1 线性表的基本概念
- 线性表(Linear List)是其组成元素间具有线性关系的一种线性结构，是由n个具有相同数据类型的数据元素a0、a1、…、an-1构成的有限序列，一般表示为：
- (a0,ai,…ai,ai+1,…,an-1)
- 其中，数据元素ai可以是字母、整数、浮点数、对象或其他更复杂的信息，i代表数据元素在线性表中的位序号(0≤i<n)，n是线性表的元素个数，称为线性表的长度，当n=0时线性表为空表。

2.1 线性表及其基本操作



- 例如：英文字母表(A,B,…,Z)是一个表长为26的线性表。
- 又如表2.1所示的书籍信息表，这个信息表中的所有记录序列构成了一个线性表，线性表中的每个数据元素都是由书名、作者、出版社、价格4个数据项构成的记录。

表2.1 书籍信息表

书名	作者	出版社	价格
软件工程实用教程	吕云翔	清华大学出版社	49.00

2.1 线性表及其基本操作

- 线性表中的数据元素具有线性的“一对一”的逻辑关系，是与位置有关的，即第i个元素ai处于第i-1个元素ai-1的后面和第i+1个元素ai+1的前面。这种位置上的有序性就是一种线性关系，可以用二元组表示为L=(D,R)，其中有以下关系：

$$D=\{a_i|0\leqslant i<n\}$$

$$R=\{r\}$$

$$r=\{<a_i,a_{i-1}>|0\leqslant i<n-1\}$$



2.1 线性表及其基本操作



对应的逻辑结构如图2.1所示。

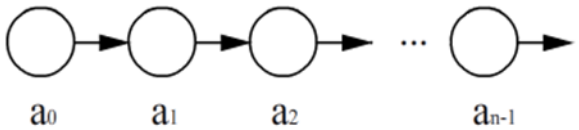


图2.1 线性表的逻辑结构图

在线性表(a0,a1,…,an-1)中，a0为开始结点，没有前驱元素，an-1为终端结点，没有后继元素。除开始结点和终端结点外，每个数据元素ai都有且仅有一个前驱元素和后继元素。

2.1 线性表及其基本操作

- **2.1.2 抽象数据类型描述**

线性表的抽象数据Python描述如下：

```
from abc import ABCMeta,abstractmethod,abstractproperty

class IList(metaclass=ABCMeta):
        @abstractmethod
        def clear(self):
                pass
        @abstractmethod
        def isEmpty(self):
                pass
        @abstractmethod
        def length(self):
                pass
        @abstractmethod
        def get(self,i):
                pass
        @abstractmethod
        def insert(self,x):
                pass
        @abstractmethod
        def remove(self,i):
                pass
        @abstractmethod
        def indexOf(self,x):
                pass
        @abstractmethod
        def display(self):
                pass
```

2.1 线性表及其基本操作

【例2.1】有线性表A=(1,2,3,4,5,6,7)，求length()、isEmpty()、get(3)、indexOf(4)、display()、insert(2,7)和remove(4)等基本运算的执行结果。

解：

length()=7；

isEmpty()返回false；

get(3)返回4；

indexOf(4)返回3；

display()输出1,2,3,4,5,6,7；

insert(2,7)执行后线性表A变为1,2,7,3,4,5,6,7；

remove(4)执行后线性表A变为1,2,3,4,6,7。

2.1 线性表及其基本操作

- **2.1.3 线性表的存储和实现**

在2.1.2节中，线性表的抽象数据Python抽象类包含了线性表的主要基本操作，如果要使用这个接口，还需要具体的类来实现。线性表的Python抽象类的实现方法主要有以下两种。

(1) 基于顺序存储的实现。

(2) 基于链式存储的实现。

2.2 线性表的顺序存储

- **2.2.1 顺序表**
- 1. 定义

线性表的顺序存储结构是把线性表中的所有元素按照其逻辑顺序依次存储到计算机的内存单元中指定存储位置开始的一块连续的存储空间中，称为顺序表。顺序表用一组连续的内存单元依次存放数据元素，元素在内存中的物理存储次序和它们在线性表中的逻辑次序一致，即元素a_i与其前驱元素a_{i-1}和后继元素a_{i+1}的存储位置相邻，如图2.2所示。

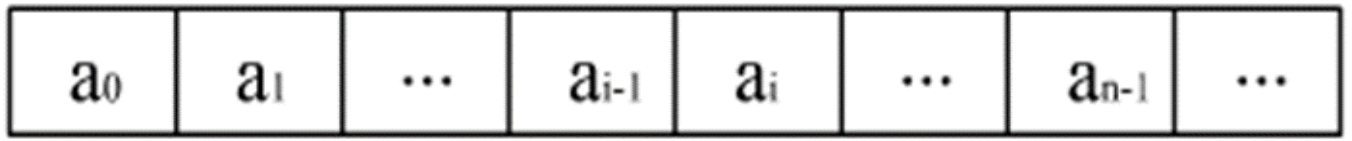


图2.2 顺序表

2.2 线性表的顺序存储

又因为数据表中的所有数据元素具有相同的数据类型，所以只要知道顺序表的基地址和数据元素所占存储空间的大小即可计算出第i个数据元素的地址，可表示为：

$$Loc(a_i)=Loc(a_0)+i\times c，其中0\leqslant i\leqslant n-1$$

其中，$Loc(a_i)$是数据元素a_i的存储地址，$Loc(a_0)$是数据元素a_0的存储地址，即顺序表的基地址，i为元素位置，c为一个数据元素占用的存储单元。

可以看出，计算一个元素地址所需的时间为常量，与顺序表的长度n无关；存储地址是数据元素位序号i的线性函数。因此，存取任何一个数据元素的时间复杂度为O(1)，顺序表是按照数据元素的位序号随机存取的结构。

2.2 线性表的顺序存储

- 2. 特点

(1) 在线性表中逻辑上相邻的元素在物理存储位置上也同样相邻。

(2) 可按照数据元素的位序号进行随机存取。

(3) 进行插入、删除操作需要移动大量的数据元素。

(4) 需要进行存储空间的预先分配，可能会造成空间浪费，但存储密度较高。

2.2 线性表的顺序存储

- 3. 描述

可以使用数组来描述线性表的顺序存储结构。在程序设计语言中数组是一种构造数据类型。数组存储具有相同数据类型的元素集合，数组的存储单元个数称为数组长度，每个存储单元的地址是连续的，每个元素连续存储。数组通过下标识别元素，元素的下标是其存储单元序号，表示元素在数组中的位置。一维数组使用一个下标唯一确定一个元素，二维数组使用两个下标唯一确定一个元素。

2.2 线性表的顺序存储

下面是顺序表类的Python语言描述：

```
1  class SqList(IList): ↵
2          def __init__(self,maxsize): ↵
3                  self.curLen = 0 # 顺序表的当前长度 ↵
4                  self.maxSize = maxsize # 顺序表的最大长度 ↵
5                  self.listItem = [None] * self.maxSize # 顺序表储存空间 ↵
6          def clear(self): ↵
7                  '''将线性表置成空表''' ↵
8                  self.curLen = 0 ↵
9          def isEmpty(self): ↵
10                 '''判断线性表是否为空表''' ↵
11                 return self.curLen == 0 ↵
```

2.2 线性表的顺序存储

```
12          def length(self):
13                  '''返回线性表的长度'''
14                  return self.curLen
15          def get(self,i):
16                  '''读取并返回线性表中的第 i 个数据元素'''
17                  if i<0 or i>self.curLen-1:
18                          raise Exception("第 i 个元素不存在")
19                  return self.listItem[i]
20          def insert(self,i,x):
21                  '''插入 x 作为第 i 个元素'''
22                  if self.curLen == self.maxSize:
23                          raise Exception("顺序表满")
24                  if i<0 or i>self.curLen:
25                          raise Exception("插入位置非法")
```

2.2 线性表的顺序存储

```
26                for j in range(self.curLen,i-1,-1): ↵
27                        self.listItem[j]=self.listItem[j-1] ↵
28                self.listItem[i] = x ↵
29                self.curLen += 1 ↵
30        def remove(self,i): ↵
31                '''删除第 i 个元素''' ↵
32                if i<0 or i>self.curLen-1: ↵
33                        raise Exception("删除位置非法") ↵
34                for j in range(i,self.curLen): ↵
35                        self.listItem[j] = self.listItem[j+1] ↵
36                self.curLen -= 1 ↵
```

2.2 线性表的顺序存储

```
37          def indexOf(self,x): ↵
38                  '''返回元素 x 首次出现的位序号''' ↵
39                  for i in range(self.curLen): ↵
40                          if self.listItem[i] == x: ↵
41                                  return i ↵
42                  return -1 ↵
43          def display(self): ↵
44                  '''输出线性表中各个数据元素的值''' ↵
45                  for i in range(self.curLen): ↵
46                          print(self.listItem[i],end=' ') ↵
```

2.2 线性表的顺序存储

- **2.2.2 顺序表的基本操作实现**
- 1. 插入操作

插入操作insert(i,x)是在长度为n的顺序表的第i个数据元素之前插入值为x的数据元素，其中0≤i≤n，当i=0时在表头插入，当i=n时在表尾插入，在插入操作完成后表长加1，顺序表的逻辑结构由$(a_0,a_1,\cdots,a_{i-1},a_i,\cdots,a_{n-1})$变成了$(a_0,a_1,\cdots,a_{i-1},x,a_i,\cdots,a_{n-1})$，如图2.3所示。

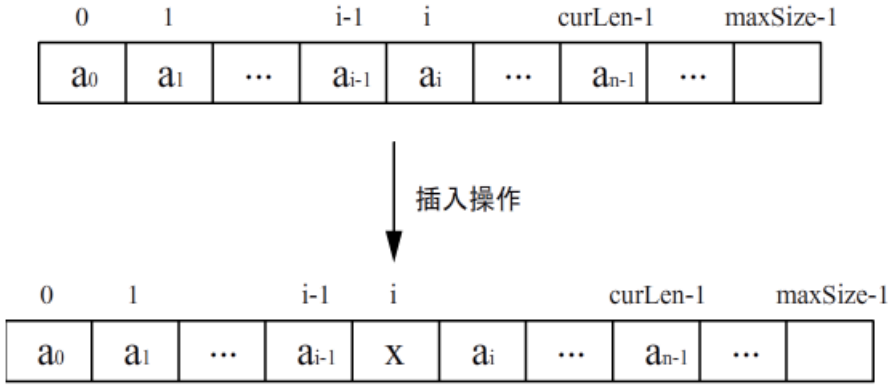


图2.3插入操作前后的顺序表存储结构图

2.2 线性表的顺序存储

- 其主要步骤如下。

(1) 判断顺序表的存储空间是否已满，若已满则抛出异常。

(2) 判断参数i的值是否满足0≤i≤curLen，若不满足则抛出异常。

(3) 将插入位置及其之后的所有数据元素后移一个存储位置。

(4) 在位置i处插入新的数据元素x。

(5) 表长加1。

2.2 线性表的顺序存储

【算法2.1】顺序表的插入操作算法。

```
1  def insert(self,i,x): ↵
2          '''插入 x 作为第 i 个元素''' ↵
3          if self.curLen == self.maxSize: # 判断顺序表的存储空间是否已满 ↵
4                  raise Exception("顺序表满") ↵
5          if i<0 or i>self.curLen: # 判断参数的值是否满足 ↵
6                  raise Exception("插入位置非法") ↵
7          for j in range(self.curLen,i-1,-1): ↵
8                  self.listItem[j]=self.listItem[j-1] # 将插入位置及之后的数据元素后移一个存储位置 ↵
9          self.listItem[i] = x # 在位置处插入新的数据元素 ↵
10         self.curLen += 1 # 表长加 1 ↵
```

2.2 线性表的顺序存储

- 算法的时间复杂度分析：
- 分析算法可以看出，在对顺序表进行插入操作时，时间花费主要用于第8行的数据元素的移动上。假设顺序表的表长为n，若插入在表头，则需要移动n个元素，若插入在表尾，则需要移动0个元素。设插入位置为i，则第8行语句的执行次数为n-i。所以每次插入操作数据元素的平均移动次数为：

$$\sum_{i=0}^{n} p_i(n-i)$$

- 其中，p_i是在顺序表的第i个存储位置插入数据元素的概率，假设每个插入位置出现的概率相同，即为$\frac{1}{n+1}$，可得：

$$\frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{n}{2}$$

- 即在等概率情况下，插入一个数据元素平均需要移动顺序表数据元素的一半，算法的时间复杂度为O(n)。

2.2 线性表的顺序存储

【例2.2】设A是一个线性表($a_0,a_1,\cdots,a_n$)，采用顺序存储结构，则在等概率的前提下平均每插入一个元素需要移动的元素个数为多少？若元素插在a_i与a_{i+1}之间($1\leqslant i\leqslant n$)的概率为$\frac{n-i}{\frac{n(n+1)}{2}}$，则平均每插入一个元素所要移动的元素个数又是多少？

2.2 线性表的顺序存储

- 解:
- 分析可得:

$$\sum_{i=1}^{n+1}(n-i+1)=\frac{n}{2}$$

$$\sum_{i=1}^{n}\frac{(n-i)^2}{n(n+1)/2}=(2n+1)/3$$

2.2 线性表的顺序存储

- 2. 删除操作
- 删除操作remove(i,x)是将长度为n的顺序表的第i个数据元素删除，其中0≤i≤n-1，删除操作完成后表长减1，顺序表的逻辑结构由$(a_0,a_1,\cdots,a_{i-1},a_i,\cdots,a_{n-1})$变成了$(a_0,a_1,\cdots,a_{i-1},a_{i+1},\cdots,a_{n-1})$，如图2.4所示。

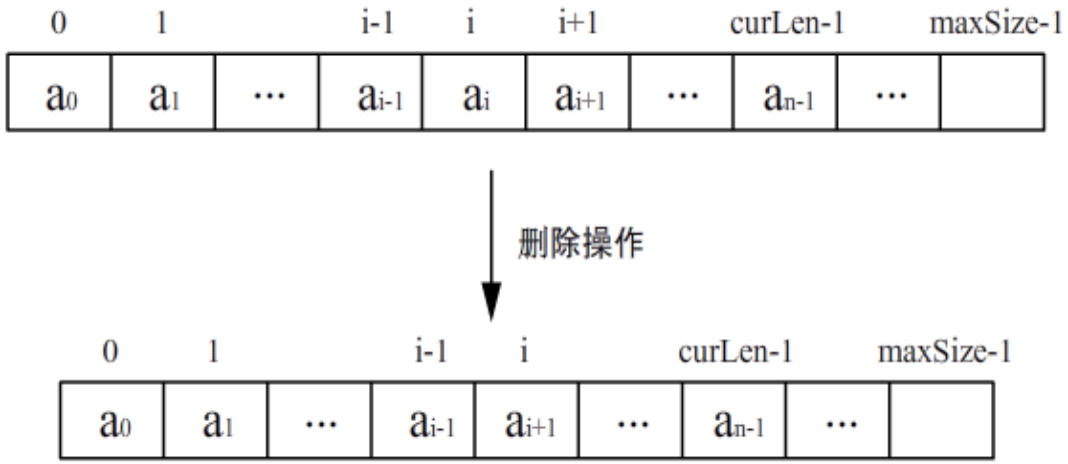


图2.4删除操作前后的顺序表存储结构图

2.2 线性表的顺序存储

- 其主要步骤如下。

(1) 判断参数i是否满足0≤i≤curLen-1，若不满足则抛出异常。

(2) 将第i个数据元素之后的数据元素都向前移动一个存储单元。

(3) 表长减1。

2.2 线性表的顺序存储

- 【算法2.2】顺序表的删除操作算法。

```
1  def remove(self,i):
2          '''删除第 i 个元素'''
3          if i<0 or i>self.curLen-1:
4                  raise Exception("删除位置非法")
5          for j in range(i,self.curLen):
6                  self.listItem[j] = self.listItem[j+1]
7          self.curLen -= 1
```

2.2 线性表的顺序存储

- 算法的时间复杂度分析:
- 分析算法可以看出，删除数据元素的时间花费主要用于第6行的数据元素的移动上。若顺序表的表长为n，删除表头数据元素需要移动n个数据元素，删除表尾数据元素需要移动0个数据元素。设删除位置为i，则第6行语句的执行次数为n-i-1。所以每次删除操作数据元素的平均移动次数为:

$$\sum_{i=0}^{n-1} p_i(n-i-1)$$

- 其中，p_i是删除顺序表的第i个存储位置的数据元素的概率，假设每个删除位置出现的概率相同，即为1n，可得:

$$\frac{1}{n}\sum_{i=0}^{n-1}(n-i-1)=\frac{n-1}{2}$$

- 即在等概率情况下，删除一个数据元素平均需要移动的顺序表的数据元素为$\frac{n-1}{2}$个，算法的时间复杂度为O(n)。

2.2 线性表的顺序存储

- 3. 查找操作
- 查找操作indexOf(x)是在长度为n的顺序表中寻找初次出现的数据元素值为x的数据元素的位置。
- 其主要步骤为将x与顺序表中的每一个数据元素的值进行比较，若相等，则返回该数据元素的位置；若比较结束未找到等值的数据元素，返回-1。

2.2 线性表的顺序存储

- 【算法2.3】顺序表的查找操作算法。

```
1  def indexOf(self,x): ↵
2      '''返回元素 x 首次出现的位序号''' ↵
3      for i in range(self.curLen): ↵
4          if self.listItem[i] == x:↵
5              return i ↵
6      return -1 ↵
```

2.2 线性表的顺序存储

- 算法的时间复杂度分析:
- 查找操作的比较次数取决于元素位置。分析算法可以看出，查找操作的时间花费主要集中在数据元素的比较上，设顺序表的数据元素个数为n，则比较次数最少为1、最多为n。假设各数据元素的查找概率相等，则数据元素的平均比较次数为:

$$\sum_{i=1}^{n} \frac{i}{n} = \frac{n+1}{2}$$

- 即在等概率情况下，查找一个数据元素平均需要比较的顺序表的数据元素为$\frac{n+1}{2}$个，算法的时间复杂度为O(n)。

2.2 线性表的顺序存储

【例2.3】建立一个由a~z的26个字母组成的字母顺序表，求每个字母的直接前驱和直接后继，编程实现。

```
L = SqList(26)
for i in range(26):
        L.insert(i,chr(ord('a')+i))

while True:
        i = input("请输入需要查询元素的位序号:\n")
        i = int(i)
        if i>0 and i<25:
                print("第%s个元素的直接前驱为:%s" % (i,L.get(i-1)))
                print("第%s个元素的直接后继为:%s" % (i,L.get(i+1)))
        elif i==0:
                print("第%s个元素的直接前驱不存在" % (i,))
                print("第%s个元素的直接后继为:%s" % (i,L.get(i+1)))
        elif i==25:
                print("第%s个元素的直接前驱为:%s" % (i,L.get(i-1)))
                print("第%s个元素的直接后继不存在" % (i,))
        else:
                print("位置非法")
```

2.2 线性表的顺序存储

【例2.4】建立一个顺序表，表中数据为5个学生的成绩(89,93,92,90,100)，然后查找成绩为90的数据元素，并输出其在顺序表中的位置。

```
1  q = SqList(5)
2  for i,x in zip(range(5),[89,93,92,90,100]):
3          q.insert(i,x)
4  res = q.indexOf(90)
5  if res==-1:
6          print("顺序表中不存在成绩为 90 的数据元素")
7  else:
8          print("顺序表中成绩为 90 的数据元素的位置为:%s" % res)
```

2.2 线性表的顺序存储

- 综上所述，顺序表具有较好的静态特性、较差的动态特性。
- （1） 顺序表利用元素的物理存储次序反映线性表元素的逻辑关系，不需要额外的存储空间进行元素间关系的表达。顺序表是随机存储结构，存取元素a_i的时间复杂度为O(1),并且实现了线性表抽象数据类型所要求的基本操作。
- （2） 插入和删除操作的效率很低，每插入或删除一个数据元素，元素的移动次数较多，平均移动顺序表中数据元素个数的一半；并且数组容量不可更改，存在因容量小造成数据溢出或者因容量过大造成内存资源浪费的问题。

2.3 线性表的链式存储和实现

采用链式存储方式存储的线性表称为链表，链表是用若干地址分散的存储单元存储数据元素，逻辑上相邻的数据元素在物理位置上不一定相邻，必须采用附加信息表示数据元素之间的逻辑关系，因此链表的每一个结点不仅包含元素本身的信息　数据域，而且包含元素之间逻辑关系的信息，即逻辑上相邻结点地址的指针域。

2.3 线性表的链式存储和实现

- **2.3.1 单链表**
- 单链表是指结点中只包含一个指针域的链表，指针域中储存着指向后继结点的指针。单链表的头指针是线性表的起始地址，是线性表中第一个数据元素的存储地址，可作为单链表的唯一标识。单链表的尾结点没有后继结点，所以其指针域值为None。
- 为了使操作简便，在第一个结点之前增加头结点，单链表的头指针指向头结点，头结点的数据域不存放任何数据，指针域存放指向第一个结点的指针。空单链表的头指针head为None。图2.5为不带头结点的单链表的存储示意图，图2.6为带头结点的单链表的存储示意图。

2.3 线性表的链式存储和实现

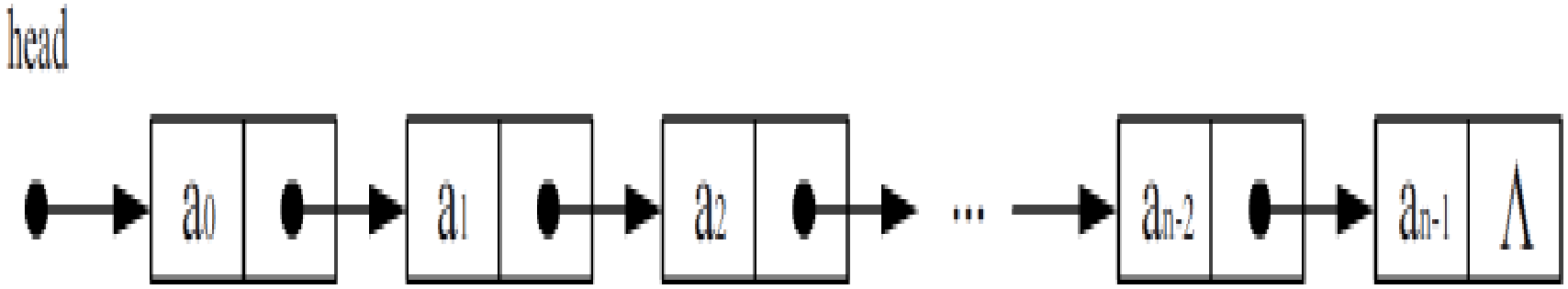


图2.5不带头结点的单链表

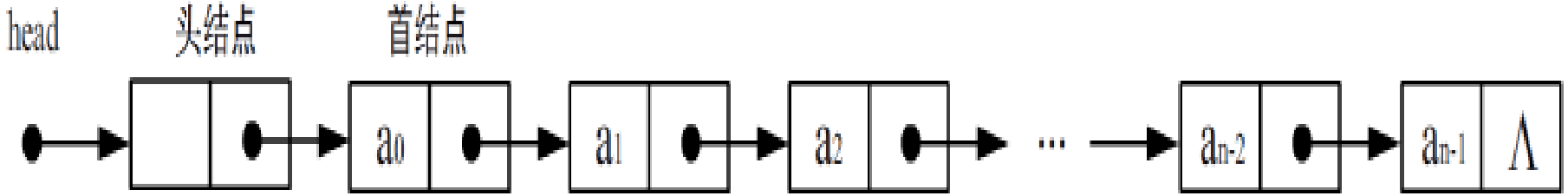


图2.6带头结点的单链表

2.3 线性表的链式存储和实现

单链表的结点的存储空间是在插入和删除过程中动态申请和释放的，不需要预先分配，从而避免了顺序表因存储空间不足需要扩充空间和复制元素的过程，避免了顺序表因容量过大造成内存资源浪费的问题，提高了运行效率和存储空间的利用率。

2.3 线性表的链式存储和实现

1.结点类描述

```
1  class Node(object):
2          def __init__(self,data=None,next=None):
3                  self.data = data
4                  self.next = next
```

2.3 线性表的链式存储和实现

2.单链表类描述

```
1  class LinkList(IList): ↵
2          def __init__(self): ↵
3                  self.head = Node() # 构造函数初始化头结点↵
4      ↵
5          def create(self,l,order): ↵
6                  if order: ↵
7                          self.create_tail(l) ↵
8                  else: ↵
9                          self.create_head(l) ↵
```

2.3 线性表的链式存储和实现

```
11      def create_tail(self,l):
12          pass
13
14      def create_head(self,l):
15          pass
16
17      def clear(self):
18          '''将线性表置成空表'''
19          self.head.data = None
20          self.head.next = None
21
22      def isEmpty(self):
23          '''判断线性表是否为空表'''
24          return self.head.next == None
```

```
26      def length(self):
27          '''返回线性表的长度'''
28          p = self.head.next
29          length = 0
30          while p is not None:
31              p = p.next
32              length += 1
33          return length
34
35      def get(self,i):
36          '''读取并返回线性表中的第 i 个数据元素'''
37          pass
38
39      def insert(self,i,x):
40          '''插入 x 作为第 i 个元素'''
41          pass
```

2.3 线性表的链式存储和实现

```
43          def remove(self,i):
44                  '''删除第 i 个元素'''
45                  pass
46
47          def indexOf(self,x):
48                  '''返回元素 x 首次出现的位序号'''
49                  pass
50
51          def display(self):
52                  '''输出线性表中各个数据元素的值'''
53                  p = self.head.next
54                  while p is not None:
55                          print(p.data,end=' ')
56                          p = p.next
```

2.3 线性表的链式存储和实现

- **2.3.2 单链表的基本操作实现**
- 1. 查找操作

(1) 位序查找get(i)是返回长度为n的单链表中第i个结点的数据域的值，其中0≤i≤n-1。由于单链表的存储空间不连续，因此必须从头结点开始沿着后继结点依次进行查找。

2.3 线性表的链式存储和实现

【算法2.4】位序查找算法。

```
1  def get(self,i): ↵
2          '''读取并返回线性表中的第 i 个数据元素''' ↵
3          p = self.head.next # p 指向单链表的首结点 ↵
4          j = 0 ↵
5          # 从首结点开始向后查找，直到 p 指向第 i 个结点或者 p 为 None
6          while j<i and p is not None: ↵
7                  p = p.next ↵
8                  j += 1 ↵
9          if j>i or p is None: # i 不合法时抛出异常 ↵
10                 raise Exception("第"+i+"个数据元素不存在") ↵
11         return p.data ↵
```

2.3 线性表的链式存储和实现

- (2) 查找操作indexOf(x)是在长度为n的单链表中寻找初次出现的数据域值为x的数据元素的位置。

其主要步骤为将x与单链表中的每一个数据元素的数据域进行比较，若相等，则返回该数据元素在单链表中的位置； 若比较结束未找到等值的数据元素，返回-1。

2.3 线性表的链式存储和实现

【算法2.5】按值查找。

```
1  def indexOf(self,x): ↵
2          '''返回元素 x 首次出现的位序号''' ↵
3          p = self.head.next ↵
4          j = 0 ↵
5          while p is not None and not (p.data == x):
6                  p = p.next ↵
7                  j += 1 ↵
8          if p is not None: ↵
9                  return j ↵
10         else: ↵
11                 return -1 ↵
```

2.3 线性表的链式存储和实现

- 2. 插入操作
- 插入操作insert(i,x)是在长度为n的单链表的第i个结点之前插入数据域值为x的新结点，其中0≤i≤n，当i=0时，在表头插入，当i=n时在表尾插入。
- 与顺序表相比，单链表不需要移动一批数据元素，而只需要改变结点的指针域，改变有序对，即可实现数据元素的插入，即$<a_{i-1},a_i>$转变为$<a_{i-1},x>$和$<x,a_i>$，如图2.7所示。

2.3 线性表的链式存储和实现

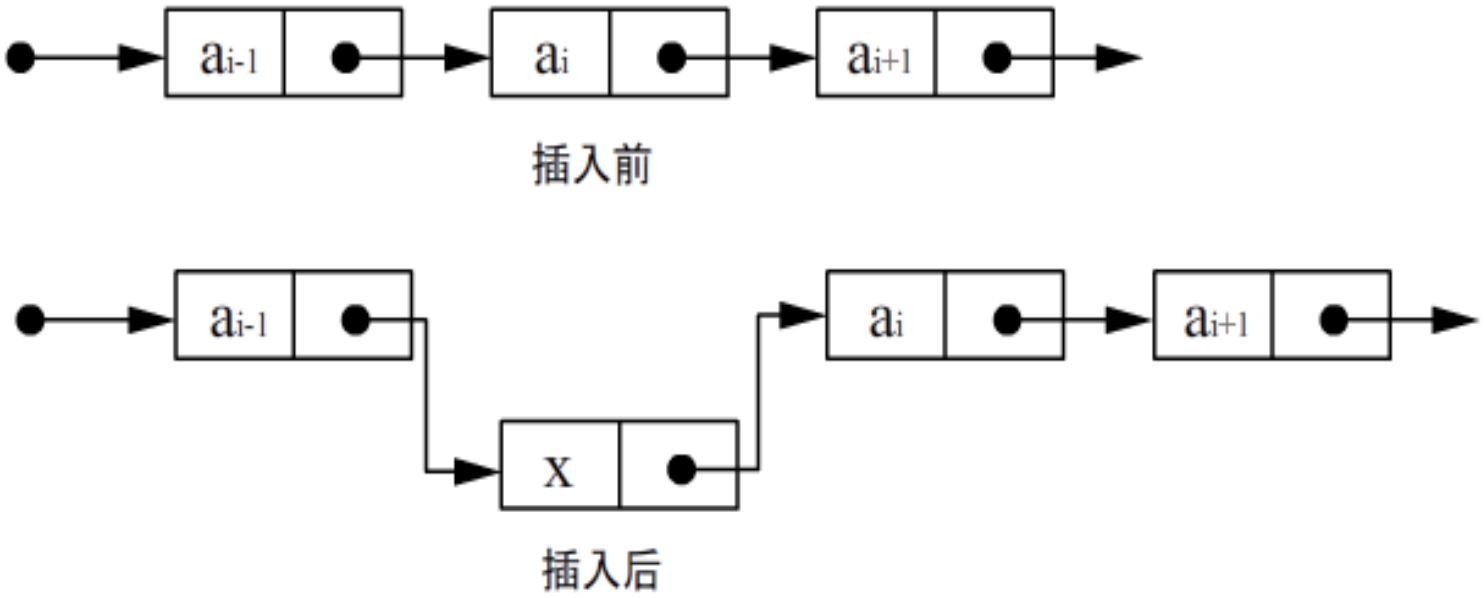


图2.7单链表上的插入

2.3 线性表的链式存储和实现

- 插入操作的主要步骤如下：

(1) 查找到插入位置的前驱结点，即第i-1个结点。

(2) 创建数据域值为x的新结点。

(3) 修改前驱结点的指针域为指向新结点的指针，新结点的指针域为指向原第i个结点的指针。

2.3 线性表的链式存储和实现

【算法2.6】带头结点的单链表的插入操作。

```
1  def insert(self,i,x):
2      '''插入 x 作为第 i 个元素'''
3      p = self.head
4      j = -1
5      while p is not None and j < i-1:
6          p = p.next
7          j += 1
8      if j>i-1 or p is None:
9          raise Exception("插入位置不合法")
10     s = Node(x,p.next)
11     p.next = s
```

2.3 线性表的链式存储和实现

【算法2.7】不带头结点的单链表的插入操作。

```
1  def insert(self,i,x): ↵
2          p = self.head ↵
3          j = -1 ↵
4          while p is not None and j < i-1: ↵
5                  p = p.next ↵
6                  j += 1 ↵
7          if j>i-1 or p is None: ↵
8                  raise Exception("插入位置不合法")
9          s = Node(data=x) ↵
10         if i==0: ↵
11                 s.next = self.head ↵
12         else: ↵
13                 s.next = p.next ↵
14                 p.next = s ↵
```

2.3 线性表的链式存储和实现

- 分析以上代码可以发现，由于链式存储采用的是动态存储分配空间，所以在进行插入操作之前不需要判断存储空间是否已满。
- 在带头结点的单链表上进行插入操作时，无论插入位置是表头、表尾还是表中，操作语句都是一致的；但是在不带头结点的单链表上进行插入操作时，在表头插入和在其他位置插入新结点的语句是不同的，需要分两种情况进行处理。

2.3 线性表的链式存储和实现

- 3. 删除操作
- 删除操作remove(i,x)是将长度为n的单链表的第i个结点删除，其中0≤i≤n-1。
- 与顺序表相比，单链表不需要移动一批数据元素，而只需要改变结点的指针域，实现有序对的改变，即可删除结点，即$<a_{i-1},a_i>$和$<a_i,a_{i+1}>$转变为$<a_{i-1},a_{i+1}>$，如图2.8所示。

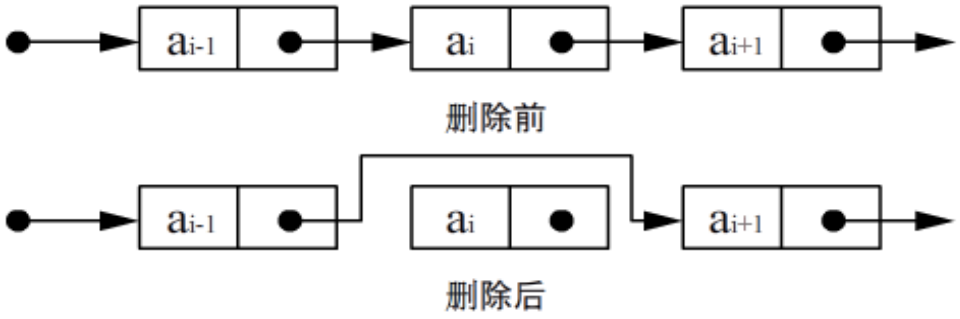


图2.8单链表的删除操作

2.3 线性表的链式存储和实现

- 其主要步骤如下。

  (1) 判断单链表是否为空。

  (2) 查找待删除结点的前驱结点。

  (3) 修改前驱结点的指针域为待删除结点的指针域。

2.2 线性表的顺序存储

【算法2.8】删除操作。

```
1  def remove(self,i): ↵
2          '''删除第 i 个元素''' ↵
3          p = self.head ↵
4          j = -1 ↵
5          # 寻找第 i 个结点的前驱结点 ↵
6          while p is not None and j<i-1: ↵
7                  p = p.next ↵
8                  j += 1 ↵
9          if j>i-1 or p.next is None: ↵
10                 raise Exception("删除位置不合法")
11         p.next = p.next.next ↵
```

2.3 线性表的链式存储和实现

- 4. 单链表的建立操作

  1) 头插法

  将新结点插入到单链表的表头，读入的数据顺序与结点顺序相反。

【算法2.9】头插法。

2.3 线性表的链式存储和实现

- 2) 尾插法

  将新结点插入到单链表的表尾，读入的数据顺序与结点顺序相同。

【算法2.10】尾插法。

```
1  def create_tail(self,l):
2          for item in l:
3                  self.insert(self.length(),item)
```

2.3 线性表的链式存储和实现

- 【例2.5】编程实现将列表中的元素构建成一个有序的单链表。

```
1  data = [i for i in range(10)]
2  ll = LinkList()
3  ll.create(data,True)
4  ll.display()
```

2.3 线性表的链式存储和实现

- **2.3.3 其他链表**

1. 循环链表

循环链表与单链表的结构相似，只是将链表的首尾相连，即尾结点的指针域为指向头结点的指针，从而形成了一个环状的链表。

循环链表与单链表的操作算法基本一致，判定循环链表中的某个结点是否为尾结点的条件不是它的后继结点为空，而是它的后继结点是否为头结点。

在实现循环链表时可用头指针或尾指针或二者同时使用来标识循环链表，通常使用尾指针来进行标识，可简化某些操作。

2.3 线性表的链式存储和实现

- 2. 双向链表

双向链表的结点具有两个指针域，一个指针指向前驱结点，一个指针指向后继结点。使得查找某个结点的前驱结点不需要从表头开始顺着链表依次进行查找，减小时间复杂度。

1）结点类描述

```
1  class DuLNode(object):
2      def __init__(self,data=None,prior=None,next=None):
3          self.data = data
4          self.prior = prior
5          self.next = next
```

2.3 线性表的链式存储和实现

- 2) 双向链表的基本操作实现

其与单链表的不同之处主要在于进行插入和删除操作时每个结点需要修改两个指针域。

【算法2.11】插入操作。

```
def insert(self,i,x):
        p = self.head
        j = -1
        # 寻找插入位置 i
        while p is not None and j<i:
                p = p.next
                j += 1
        if j>i or p is None:
                raise Exception("插入位置不合法")
        s = DuLNode(data=x)
        p.prior.next = s
        s.next = p
        p.prior = s
```

2.3 线性表的链式存储和实现

【算法2.12】删除操作。

```
1  def remove(self,i): ↵
2          p = self.head ↵
3          j = -1 ↵
4          while p is not None and j<i: ↵
5                  p = p.next ↵
6                  j += 1 ↵
7          if j>i or p is None: ↵
8                  raise Exception("删除位置不合法")
9          p.prior.next = p.next ↵
10         p.next.piror = p.piror ↵
```

2.4 线性表与链表的比较

- 顺序表与链表的比较如表2.2所示。

表2.2顺序表与链表的比较

	顺序表	链表
优点	（1） 可进行高效随机存取； （2） 存储密度高，空间开销小； （3） 实现简单，便于使用	（1） 灵活，可进行存储空间的动态分配； （2） 插入、删除效率高
缺点	（1） 需要预先分配存储空间； （2） 不便于进行插入和删除操作	（1） 存储密度低； （2） 不可按照位序号随机存取

小结

- (1) 线性表是其组成元素间具有线性关系的一种线性结构，其实现方式主要为基于顺序存储的实现和基于链式存储的实现。
- (2) 线性表的顺序存储结构称为顺序表，可用数组实现，可对数据元素进行随机存取，时间复杂度为O(1)，在插入或删除数据元素时时间复杂度为O(n)。
- (3) 线性表的链式存储结构称为链表，不能直接访问给定位置上的数据元素，必须从头结点开始沿着后继结点进行访问，时间复杂度为O(n)。在插入或删除数据元素时不需要移动任何数据元素，只需要更改结点的指针域即可，时间复杂度为O(1)。
- (4) 循环链表将链表的首尾相连，即尾结点的指针域为指向头结点的指针，从而形成了一个环状的链表。
- (5) 双向链表的结点具有两个指针域，一个指针指向前驱结点，一个指针指向后继结点，使得查找某个结点的前驱结点不需要从表头开始顺着链表依次进行查找，减小时间复杂度。

谢谢观看